TP8 - Food Store en la feria

1.a Qué datos viajan a la tablet

La tablet recibe una instantánea del catálogo (vista `v_snapshot_movil`, exportada a CSV). Todo lo que no se usa para armar un borrador o evaluar una solicitud se queda en el servidor.

Dato	Viaja	Motivo
`id`	viaja	identifica el producto en el borrador
`nombre`	viaja	para mostrar el catálogo sin conexión
`precio_centavos`	viaja	precio actual, en enteros, para detectar conflictos
`stock`	viaja	para rechazar cantidades que no alcanzan
`disponible`	viaja	derivado: activo y con stock
`eliminado`	viaja	derivado: producto dado de baja (lápida)
`copiado_en`	viaja	fecha de la copia
`filas_copia`	viaja	total esperado, para verificar la importación
`cliente` completo	no viaja	datos personales: la tablet no los necesita
`created_at`, `categoria_id`	no viaja	no se usan en la tablet
pedidos y detalles del servidor	no viaja	son las ventas; las administra solo el servidor

La fuente de verdad es PostgreSQL: el precio, el stock y las ventas los decide el servidor. La tablet trabaja con una copia de solo lectura que puede estar vieja, y por eso cada solicitud se vuelve a comparar contra una instantánea nueva. Lo único que manda la tablet son sus borradores y solicitudes, y nada de eso vuelve al servidor en este TP.

2.c Casos de sincronización

Todo se corre con `bash scripts/correr_casos.sh` (desde `TP8/`). Para cada caso, el script arma el borrador en SQLite, cambia la **copia** del servidor (`food_store_copia`) si hace falta, exporta una instantánea nueva, la importa y corre `movil/semaforo.sql`. Cada evaluación guarda en `cola_sync.snapshot_copiado_en` la instantánea que usó: S0 del `2026-11-01T23:14:07Z` (C5 y C6), la de después de subir el precio del `23:14:09Z` (C2 y C3) y la de después de la baja del `23:14:12Z` (C4). Resultado final de `cola_sync`:

Caso	Qué se preparó (comando)	Resultado (estado · motivo)	Por qué
C5	Borrador de 2 unidades del producto D, sin cambios (`pruebas_sqlite.sql`) → `semaforo.sql`	`SIMULADO_APROBADO`	Precio y stock vistos coinciden con la instantánea. Es solo una simulación: no crea un pedido en el servidor.
C6	Reenviar la misma solicitud (`movil/casos/c5_c6.sql`) y volver a correr el semáforo	`UNIQUE constraint failed: cola_sync.solicitud_uuid` · el semáforo reporta `changes() = 0`	El UUID es la clave primaria de `cola_sync`: el reintento no se duplica. Como solo se evalúan las `PENDIENTE`, repetir la simulación no reevalúa nada.
C2	Borrador con A a 2750,00 (`c2_borrador.sql`); en el servidor `sql/casos/preparar_c2.sql` sube A a 2990,00; nueva instantánea; `semaforo.sql`	`CONFLICTO` · producto 1: precio visto 275000, actual 299000	No hay rojo pero cambió un precio. El cliente acepta (`c2_aceptar.sql`): se actualiza el precio visto y una **solicitud nueva** da `SIMULADO_APROBADO`; la vieja queda en `CONFLICTO` como historial.
C3	Borrador de 5 unidades del producto B, que tiene stock 4 (`c3_borrador.sql`)	`RECHAZADO` · producto 24: pide 5, stock 4	Rojo por stock. No hizo falta tocar el stock del servidor: el borrador pide más de lo que hay.
C4	Borrador con C (`c4_borrador.sql`); en el servidor `sql/casos/preparar_c4.sql` pone `activo = false`; nueva instantánea	`RECHAZADO` · producto 3: dado de baja	Rojo por baja lógica. Al recargar, la fila de C sigue en `producto_cache` con `eliminado = 1` (lápida), pero ya no aparece en el

			catálogo disponible (consulta C1: 0 filas para el id 3).

PostgreSQL no cambió. Al final, `sql/casos/restaurar_casos.sql` devuelve A a 2750,00 y C a activo, y `sql/verificacion_final.sql` repite las métricas de `sql/linea_base.sql` (tomada antes de tocar nada). El script las compara y falla si difieren:

Métrica	Línea base	Después de los casos
Pedidos (`count(*) FROM pedido`)	200 002	200 002
Stock total (`sum(stock) FROM producto`)	5 012 740	5 012 740
Huella md5 de `producto` (id, nombre, precio, stock, activo, categoría)	`3dfee1ff…aa1`	`3dfee1ff…aa1`

La tablet nunca se conectó a PostgreSQL (el único canal es el CSV) y ningún `SIMULADO_APROBADO` generó un pedido. Los únicos cambios en el servidor fueron los que preparan C2 y C4, hechos a mano sobre la copia y revertidos.

C6 en un sistema real (parada 8): la clave primaria de la tablet solo evita duplicados *dentro* de la tablet. La protección real va en el servidor. Ahí se guarda cada `solicitud_uuid` procesado en una tabla con restricción `UNIQUE`, en la misma transacción que crea el pedido. Si llega un reintento, el servidor devuelve el resultado original en lugar de crear un segundo pedido.

3.a Mapa de los tres motores

Cada dato del ecosistema Food Store tiene un único motor que manda (fuente de verdad). Los demás motores mantienen copias derivadas de solo lectura o directamente no almacenan el dato.

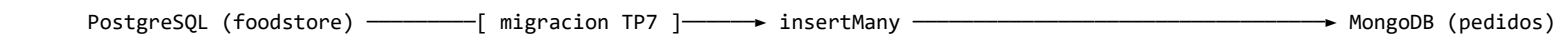
Dato	PostgreSQL (Central)	MongoDB (U7)	SQLite (Tablet feria)
Catálogo y precio actual	**manda**	no está [1]	copia (solo lectura) en `producto_cache`
Stock	**manda**	no está	copia (solo lectura) (`stock_visto`)
Pedidos confirmados (ventas)	**manda**	copia (solo lectura): colección `foodstore.pedidos` (se regenera desde PostgreSQL)	no está
Estado del pedido	**manda** (`pedido.estado` en `foodstore`)	copia (solo lectura)	no está
Clientes y datos personales	**manda**	copia (solo lectura) del subconjunto esencial dentro del pedido [2]	no está (nunca viajan a la tablet)
Instantánea CSV (`snapshot_servidor`)	**manda** (vista `v_snapshot_movil`)	no está	copia (solo lectura)
Borradores (`pedido_borrador`, `item_borrador`)	no está	no está	**manda**
Solicitudes de sincronización (`cola_sync`)	no está	no está	**manda** (la sincronización solo se simula)
Resultado del semáforo	no está	no está	**manda** (simulación local; `SIMULADO_APROBADO` nunca es una venta)

[1] Cada línea en MongoDB conserva el `precio_unitario` histórico del momento de la venta, no el precio actual del catálogo.

[2] En MongoDB se embeben los datos esenciales de cliente y producto; teléfono, dirección, activo, precio actual y stock no se embeben y pertenecen exclusivamente a PostgreSQL.

Nota de origen: MongoDB contiene 9 pedidos / 31 líneas migrados desde la base `foodstore` (TP7, que incluye `pedido.estado`); por su parte, la tablet trabaja sobre `food_store_copia`, copia de pruebas sin columna `estado`.

```
Flujo entre motores:
PostgreSQL (food_store_copia) ──[ v_snapshot_movil ]──► \copy ──► servidor_snapshot.csv ──► .import ──► SQLite (tablet)
```



3.b Plan de mantenimiento

El plan extiende el mantenimiento de la Unidad 6 (respaldo diario de PostgreSQL y prueba semanal) sin modificar el TP6:

- **Qué se respalda y restauración:**
  - *PostgreSQL:* Respaldo lógico diario con `pg_dump` y prueba de restauración semanal en servidor aislado.
  - *MongoDB:* No se respalda de forma autónoma. Se regenera desde PostgreSQL ejecutando la migración del TP7 (`insertMany`).
  - *SQLite:* Respaldo en caliente al cierre de cada jornada de feria con `VACUUM INTO 'respaldo_jornada_YYYYMMDD.db'`. La prueba de restauración se realiza semanalmente comprobando `PRAGMA integrity_check` y conteos en las cuatro tablas.
- **Contingencia ante restauración de respaldo viejo de PostgreSQL:** Las tablets no confían en su caché ni en resultados simulados previos. Invalidan `snapshot_servidor`, descargan una instantánea fresca con `copiado_en` posterior a la restauración del servidor y evalúan las solicitudes `PENDIENTE` con el semáforo. Los `SIMULADO_APROBADO` anteriores no cambian de estado (el semáforo solo evalúa `PENDIENTE`): los borradores afectados se reenvían con una solicitud nueva (UUID nuevo), que se evalúa contra la instantánea nueva.
- **Revisión periódica:** Al cierre de cada jornada de feria (y antes de la siguiente sincronización), el **Encargado del Food Truck** revisa las solicitudes en `CONFLICTO` (para actualizar precio visto o desistir) y en `RECHAZADO` (para registrar mermas o anular).

3.c Code review con IA

Se realizó una revisión técnica de los 107 scripts de la cursada completa (TP1 a TP6, Proyecto Integrador, TP7 y TP8). **Declaración de herramienta:** La revisión fue efectuada por un agente de Claude de solo lectura (modelo Claude Opus 5.5 / subagente `research`) y no por el agente Plan de OpenCode.

ID	Unidad	Categoría	Decisión	Motivo
SEC-01	TP7	Seguridad	Descartado	Los campos provienen de ENUMs estrictamente alfanuméricos en PostgreSQL (imposible inyección).
SEC-02	TP7	Seguridad	Aceptado	Validación de formato CSV en script de test local.
CORR-01	TP8	Correctitud	Descartado	La evaluación local es simulación (consistencia eventual); el stock real lo bloquea PostgreSQL.
CORR-02	TP7	Rendimiento	Aceptado	Medido con `EXPLAIN ANALYZE`: el filtro temprano baja de 621.201 filas y 2.621 buffers a 12 filas y 64 buffers, con el mismo resultado.
CORR-03	TP8	Correctitud	Descartado	El directorio en el host se llama literalmente `PosgresSQL`; corregir a ciegas rompería psql.
INTEG-01	TP8	Integridad	Aceptado	Exigencia de ejecutar `PRAGMA foreign_keys = ON;` en cada conexión cliente en SQLite.
INTEG-02	TP1–6	Integridad	Aceptado (histórico)	Falta de CHECK de subtotal en modelo relacional original (repositorio externo cerrado).
INTEG-03	TP3–4	Calidad	Aceptado (histórico)	Sesgo de 200.000 pedidos al cliente 4812 (lección aprendida de generación sintética).
INTEG-04	TP8	Integridad	Aceptado (fortaleza)	Validación estricta de formato canónico de UUIDs en tablas STRICT.
IDEM-01	TP7	Idempotencia	Aceptado	Uso de `drop()` incondicional admisible solo en entorno de laboratorio.
IDEM-02	TP7	Rendimiento	Aceptado	UPDATE masivo en PG genera tuplas muertas; requiere VACUUM ANALYZE.
IDEM-03	TP8	Idempotencia	Aceptado (fortaleza)	Patrón transaccional robusto con guarda temporal de filas y lápidas en upsert.

(La tabla completa detallada se encuentra en `docs/code_review_ia.md`).

Tres hallazgos desarrollados con comprobación empírica:

1. **[CORR-01] Supuesta "Aprobación fantasma de stock" en semáforo de TP8 (Descartado con fundamento):**
   La IA cuestionó que dos solicitudes encoladas de 2 unidades frente a un stock de 3 aprueben como simulación. Se comprobó en `evidencias/code_review_1.txt` que ambas aprueban porque `SIMULADO_APROBADO` es una propuesta desconectada monousuario en la feria, no una venta confirmada. Cada solicitud se compara contra la instantánea sin descontar nada, a propósito: descontar stock en una tablet desconectada daría inventarios distintos en cada tablet. Controlar el stock entre varias ventas le corresponde al servidor cuando la venta se confirme, y eso queda fuera del TP8.
2. **[CORR-02] Agrupación de toda la tabla en la vista comparativa del TP7 (Aceptado):**
   En `consultas_postgresql.sql`, la vista temporal `pedido_total` agrupa todo `pedido_detalle` aunque después solo se usan los 9 pedidos migrados. Se midió con `EXPLAIN (ANALYZE, BUFFERS)` la consulta d) en `food_store_copia` (script `scripts/medir_corr02.sql`; salida en `evidencias/code_review_2_explain.txt`). Con la vista original, PostgreSQL recorre las 621.201 filas: 2.621 buffers y 206,6 ms. Con el filtro `WHERE d.pedido_id IN (SELECT id FROM migrados)` lee 12 filas: 64 buffers y 0,09 ms. Las dos versiones devuelven el mismo pedido (10167). El diagnóstico de la IA era correcto, pero sus cifras (24.000 buffers, 31 filas) no estaban medidas; `evidencias/code_review_2.txt` registra las diferencias.
3. **[INTEG-01] Claves foráneas que SQLite no hace cumplir por defecto, en TP8 (Aceptado):**
   La IA advirtió que `PRAGMA foreign_keys = ON` vale por conexión y no queda guardado en el archivo `.db`. Se comprobó en `evidencias/code_review_integ01.txt` (script `scripts/demo_pragma_fk.sh`, sobre una base temporal con el esquema real): en una conexión sin el PRAGMA, insertar en `item_borrador` un `producto_id` inexistente funciona (`changes() = 1`) y solo `PRAGMA foreign_key_check` detecta el huérfano después. En otra conexión con el PRAGMA, la misma inserción falla con `FOREIGN KEY constraint failed`. Ya estaba mitigado: la misma evidencia muestra que todos los scripts de `movil/` lo activan en su primera sentencia SQL. Queda como requisito para cualquier cliente nuevo que abra la base.